BIL 463 Derleyici Tasarımı / Compiler Design

2026 Güz Dönemi – Vize Sınavı / Midterm Exam – Answer Key and Rubric

Kapsam / Coverage: Weeks 1–7 (introduction, lexical analysis, context-free grammars, top-down and bottom-up parsing)

Tarih / Date: _____ Süre / Duration: 90 dakika / minutes Toplam / Total: 100 puan / points

Instructor use only. Do not distribute before the exam. Partial credit is described per question; alternative correct answers receive full credit.

Section	Learning outcome	LO	Questions	Points	Score
A	Compiler phases and the AI connection	LO1	1–2	10	
B	Regular expressions, NFAs, DFAs	LO2	3–5	20	
C	Tokenization: maximal munch, priority	LO3	6–7	15	
D	Context-free grammars, ambiguity	LO4	8–9	15	
E	Top-down (LL(1)) parsing	LO5	10–11	20	
F	Bottom-up (LR/SLR) parsing	LO6	12–14	20	
			Total	**100**	

Section A LO1: Describe the phases of a compiler and relate them to AI systems (10 pts)

[5] 1. Which list gives the compiler phases in the correct order?

Aşağıdakilerden hangisi derleyici aşamalarını doğru sırada verir?

A. Lexical analysis → Parsing → Semantic analysis → Code generation

B. Parsing → Lexical analysis → Semantic analysis → Code generation

C. Lexical analysis → Semantic analysis → Parsing → Code generation

D. Semantic analysis → Lexical analysis → Parsing → Code generation

Answer:

A. All or nothing.

[5] 2. An LLM tokenizer (e.g., BPE) is often compared to a compiler phase. Name that phase and give **one** difference between the LLM tokenizer and a traditional one.

*Bir LLM tokenizer'ı (ör. BPE) sıklıkla bir derleyici aşamasına benzetilir. Bu aşamayı yazınız ve LLM tokenizer'ı ile geleneksel olanı arasındaki **bir** farkı belirtiniz.*

Answer:

Phase: lexical analysis (2 pts). One valid difference (3 pts), any of:

- BPE merge rules are *learned from a corpus*; lexer regex rules are *hand-written*.
- BPE works on natural language/bytes and *never rejects* input (byte-level fallback); a lexer reports an *error* on unknown input.
- BPE splits follow statistical merge order; a lexer uses maximal munch plus rule priority.

Phase correct but difference missing/wrong: 2/5. Difference correct but phase missing: 3/5.

Section B LO2: Construct regular expressions, NFAs and DFAs (20 pts)

[5] 3. Which regular expression matches an identifier: a letter followed by zero or more letters or digits?

Aşağıdaki regular expression'lardan hangisi bir harfle başlayan ve ardından sıfır veya daha fazla harf ya da rakam gelen bir identifier'ı eşleştirir?

A. `[a-z][a-z0-9]`

B. `[a-z]+[0-9]*`

C. `[a-z]([a-z]|[0-9])*`

D. `([a-z]|[0-9])+`

Answer:

C. All or nothing.

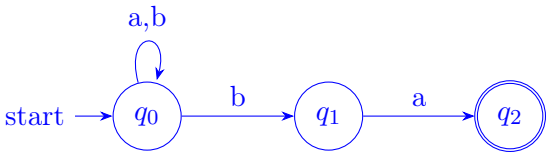
[8] 4. Write a regular expression **and** draw an NFA for the language: strings over $\{a, b\}$ that **end with "ba"**.

*Aşağıdaki dili tanıyan bir regular expression yazınız **ve** NFA çiziniz: $\{a, b\}$ alfabesi üzerinde **"ba" ile biten** string'ler.*

Answer:

Regular expression (3 pts; equivalent forms accepted): $(a|b)^*ba$

NFA (5 pts):



Rubric: start/accepting states (1); loop on q_0 for a, b (1); b-edge to q_1 (1); a-edge to accepting q_2 (1); no extra transitions changing the language (1). An NFA with ε-moves (Thompson) is accepted.

[7] 5. Convert the following NFA (strings over $\{a, b\}$ that contain "bb") to an equivalent DFA using subset construction. Rows A and B are given; **complete rows C and D**.

*Aşağıdaki NFA'yı (içinde "bb" geçen string'ler) subset construction ile eşdeğer bir DFA'ya dönüştürünüz. A ve B satırları verilmiştir; **C ve D satırlarını tamamlayınız.***

State	on a	on b
$\rightarrow q_0$ (start)	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$ (accepting)	$\{q_2\}$	$\{q_2\}$

DFA state	NFA set	on a	on b	Accepting?
A (start)	$\{q_0\}$	A	B	no
B	$\{q_0, q_1\}$	A	C	no
C	$\{q_0, q_1, q_2\}$	D	C	yes
D	$\{q_0, q_2\}$	D	C	yes

Rubric: C set (1) and its transitions (1); D set (1) and its transitions (1); accepting states C, D (1); the two remaining entries of the DFA, i.e. all transitions consistent (2). Rows A and B are given. C and D are equivalent, so the minimal DFA has 3 states; a correct minimization also earns full credit.

Section C LO3: Apply maximal munch and rule priority to tokenize input (15 pts)

[5] 6. What does the "maximal munch" rule say?

"Maximal munch" (en büyük lokma) kuralı ne söyler?

A. Choose the first token found

B. Choose the shortest matching token

C. Choose the longest possible match

D. Skip whitespace characters

Answer:

C. All or nothing.

7. A Flex-style lexer has these rules, listed in this order (whitespace is skipped):

Bir Flex tarzı lexer'ın kuralları aşağıdaki sıradadır (boşluklar atlanır):

#	Pattern	Token
1	`if`	`IF`
2	`[a-z]+`	`ID`
3	`[0-9]+`	`NUM`
4	`<=`	`LE`
5	`<`	`LT`
6	`=`	`ASSIGN`

[6] (a) List the token stream (with lexemes) produced for the input `if x<=10 iffy=7`.

`if x<=10 iffy=7` girdisi için üretilen token dizisini (lexeme'leriyle birlikte) yazınız.

Answer:

`IF("if") ID("x") LE("<=") NUM("10") ID("iffy") ASSIGN("=") NUM("7")`

Rubric: IF (1); ID(x) (1); LE, not LT then ASSIGN (1); NUM(10) (1); ID(iffy), not IF + ID(fy) (1); ASSIGN and NUM(7) (1). Token names right but lexemes missing: −1 overall.

[4] (b) Why is the lexeme `if` returned as `IF` and not `ID`, although both rules match it? What would happen if rules 1 and 2 were swapped?

`if` lexeme'i, iki kural da eşleştiği halde neden `ID` değil `IF` olarak döner? 1. ve 2. kurallar yer değiştirirse ne olur?

Answer:

Both rules match two characters, so the tie is broken by **rule priority: the rule listed first wins** (2 pts). If swapped, `if` (and every keyword) would be lexed as `ID`; the `IF` rule would never fire (2 pts). `iffy` is unaffected: maximal munch already prefers the longer `ID` match.

Section D LO4: Design context-free grammars and analyze ambiguity (15 pts)

8. Given the grammar: $E \to E + E \mid E * E \mid id$

Verilen grammar: $E \to E + E \mid E * E \mid id$

[5] (a) The leftmost derivation below for $id + id * id$ is given. Give a **different** leftmost derivation (or parse tree) of the same string, and state what this shows about the grammar.

Aşağıda $id + id * id$ *için bir leftmost derivation verilmiştir. Aynı string için* ***farklı*** *bir leftmost derivation (veya parse tree) veriniz ve bunun grammar hakkında ne gösterdiğini yazınız.*

Given: $E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$

Answer:

$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$

Rubric: second derivation valid and genuinely different, i.e. $*$ at the root (3); concludes "two parse trees for one string ⇒ the grammar is ambiguous" (2).

[5] (b) Rewrite the grammar so that it is unambiguous, $*$ binds tighter than $+$, and both operators are left-associative.

Grammar'ı belirsizliği giderilmiş, $*$ *işlemi* $+$ *işleminden önce gelecek ve her iki işlem de sol-birleşmeli (left-associative) olacak şekilde yeniden yazınız.*

Answer:

$E \to E + T \mid T \qquad T \to T * F \mid F \qquad F \to id$

Rubric: separate level for $*$ (2); left-recursive rules give left associativity (2); unambiguous and same language (1). Adding $F \to (E)$ is accepted. Right recursion ($E \to T + E$) loses 2 pts.

[5] 9. Write a context-free grammar for the language $\{a^n b^{2n} : n \geq 0\}$ (e.g., ε, abb, $aabbbb$).

$\{a^n b^{2n} : n \geq 0\}$ *dili için bir context-free grammar yazınız (ör.* ε*,* abb*,* $aabbbb$*).*

Answer:

$S \to a\, S\, bb \mid \varepsilon$

Rubric: base case ε (1); each a adds exactly two b's (2); S nested between the a and the bb (2). Equivalent grammars accepted.

Section E LO5: Construct top-down (LL(1)) parsers (20 pts)

10. Transform the grammars so they are suitable for top-down parsing:

Grammar'ları top-down parsing için uygun hale getiriniz:

[3] (a) Eliminate left recursion: $L \to L\ ,\ E \mid E$

Left recursion'ı kaldırınız: $L \to L\ ,\ E \mid E$

Answer:

$L \to E\, L' \qquad L' \to ,\ E\, L' \mid \varepsilon$ (structure 2, ε-production 1)

[3] (b) Left-factor: $S \to \texttt{if}\ E\ \texttt{then}\ S \mid \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{other}$

Left factoring uygulayınız.

Answer:

$S \to \texttt{if}\ E\ \texttt{then}\ S\ S' \mid \texttt{other} \qquad S' \to \texttt{else}\ S \mid \varepsilon$ (common prefix factored 2, new non-terminal incl. ε 1)

11. Consider the grammar of balanced parentheses: $S \to (\ S\ )\ S \mid \varepsilon$

Dengeli parantezler grammar'ını düşününüz: $S \to (\ S\ )\ S \mid \varepsilon$

[4] (a) Compute FIRST(S) and FOLLOW(S).
FIRST(S) *ve* FOLLOW(S) *kümelerini hesaplayınız.*

Answer:
FIRST(S) = { (, ε } (2) FOLLOW(S) = {) , $ } (2). Missing $ or ε: −1 each.

[5] (b) Fill in the LL(1) parsing table.
LL(1) parsing tablosunu doldurunuz.

	`(`	`)`	`$`
S	$S \to (S)S$	$S \to \varepsilon$	$S \to \varepsilon$

Rubric: 1 pt for the `(` entry, 2 pts each for `)` and `$` (ε goes under FOLLOW). Wrong column: no credit for that entry.

[5] (c) Complete the trace of the parse of `()`. The first three rows are given; **fill in the last three**.
`()` girdisinin parse izini tamamlayınız. İlk üç satır verilmiştir; ***son üç satırı doldurunuz.***

Stack (top at right)	Input	Action
`$ S`	`( ) $`	expand $S \to (S)S$
`$ S ) S (`	`( ) $`	match `(`
`$ S ) S`	`) $`	expand $S \to \varepsilon$
`$ S )`	`) $`	match `)`
`$ S`	`$`	expand $S \to \varepsilon$
`$`	`$`	accept

Rubric: match `)` (1); stack after it (1); expand $S \to \varepsilon$ chosen by lookahead `$` (2); accept (1). A propagated error loses points once.

Section F	LO6: Construct bottom-up (LR / SLR) parsers (20 pts)

[4] 12. What is a **handle** in bottom-up (shift-reduce) parsing?

Bottom-up (shift-reduce) parsing'te ***handle*** *nedir?*

Answer:

A substring of the current sentential form that matches the right-hand side of a production *and* whose reduction to the left-hand side is one step of the (reverse) rightmost derivation. Matches a RHS: 2 pts; reduction is the correct step of the rightmost derivation, not any RHS match: 2 pts.

[8] 13. Grammar: $E \to E + T \mid T$, $T \to (\ E\ ) \mid id$. The shift-reduce parse of $id + (\ id\ )$ is partly given. **Fill in the five missing rows.**

Grammar: $E \to E + T \mid T$, $T \to (\ E\ ) \mid id$. $id + (\ id\ )$ *girdisinin shift-reduce parse'ı kısmen verilmiştir.* ***Eksik beş satırı doldurunuz.***

Stack	Input	Action
`$`	`id + ( id ) $`	shift
`$ id`	`+ ( id ) $`	reduce $T \to id$
`$ T`	`+ ( id ) $`	reduce $E \to T$
`$ E`	`+ ( id ) $`	shift
`$ E +`	`( id ) $`	shift
`$ E + (`	`id ) $`	shift
`$ E + ( id`	`) $`	reduce $T \to id$
`$ E + ( T`	`) $`	reduce $E \to T$
`$ E + ( E`	`) $`	shift
`$ E + ( E )`	`$`	reduce $T \to (E)$
`$ E + T`	`$`	reduce $E \to E + T$
`$ E`	`$`	accept

Rubric: 1.6 pts per correct row (stack, input and action all correct); the inner $E \to T$ and the final $E \to E + T$ reductions are the key steps. A propagated error loses points once.

14. Consider the grammar: $S' \to S,\ \ S \to (\ S\ ),\ \ S \to x$.

Verilen grammar: $S' \to S,\ S \to (\ S\ ),\ S \to x$.

[6] (a) Complete the LR(0) automaton. States I_0, I_1 and I_3 are given; **fill in I_2, I_4 and I_5** (items and transitions).

LR(0) otomatını tamamlayınız. I_0, I_1 *ve* I_3 *durumları verilmiştir;* I_2, I_4 ***ve*** I_5 ***durumlarını*** *(item'lar ve geçişler) doldurunuz.*

State	Items	Transitions
I_0	$S' \to .S;\ S \to .(S);\ S \to .x$	$S \to I_1;\ (\to I_2;\ x \to I_3$
I_1	$S' \to S.$	(accept)
I_2	$S \to (.S);\ S \to .(S);\ S \to .x$	$S \to I_4;\ (\to I_2;\ x \to I_3$
I_3	$S \to x.$	reduce
I_4	$S \to (S.)$	$) \to I_5$
I_5	$S \to (S).$	reduce

Rubric: I_2 with closure items (2) and its three transitions incl. the self-loop on ((1); I_4 item and)-transition (2); I_5 (1).

[2] (b) Does the automaton have any shift-reduce or reduce-reduce conflicts? Briefly justify.

Otomatta shift-reduce veya reduce-reduce çakışması var mı? Kısaca gerekçelendiriniz.

Answer:

No conflicts (1). I_1, I_3, I_5 each contain a single complete item and I_0, I_2, I_4 contain only shift items, so the grammar is LR(0) (1).

Learning-outcome attainment (for outcome assessment records). Record each student's section score; "attained" at $\geq 70\%$ of a section's points.

Section	LO	Max	70% threshold
A	LO1	10	7
B	LO2	20	14
C	LO3	15	10.5
D	LO4	15	10.5
E	LO5	20	14
F	LO6	20	14

İyi çalışmalar! / Good luck!